K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

School of Engineering and Technology

Data Structures(ETCCDS202)

Assignment-4

Title: Data Management Mini Toolkit (DMMT)

Submitted By: NEHA

Roll Number: 2501730310

Program: B.Tech CSE (AI & ML)

Course: Data Structures (ETCCDS202)

Submitted To: Dr. Neetu Chauhan

1. Introduction

This project focuses on implementing important non-linear data structures used in real-world applications. The aim is to understand how data is stored, searched, and managed efficiently.

The following components were implemented:

- Binary Search Tree (BST)
- Graph (BFS and DFS traversal)
- Hash Table with collision handling

2. Objectives

- To implement BST operations (insert, search, delete, traversal)
- To represent graphs using adjacency list and perform BFS & DFS
- To implement hash table with separate chaining
- To understand real-world applications of these structures

3. Binary Search Tree (BST)

Operations Performed

- Insert elements: 50, 30, 70, 20, 40, 60, 80
- Search elements: 20, 90

- Delete cases:
  - Leaf node → 20
  - One child → 60
  - Two children → 50

Output

Search 20: True
Search 90: False
Inorder: 20 30 40 50 60 70 80
After delete 20: 30 40 50 60 70 80
After delete 60: 30 40 50 65 70 80
After delete 50: 30 40 65 70 80

Explanation

- Inorder traversal prints elements in sorted order because it follows Left → Root → Right
- Time Complexity:
  - Average: O(log n)
  - Worst: O(n)

4. Graph (BFS & DFS)

Adjacency List

A -> [('B', 2), ('C', 4)]
B -> [('D', 7), ('E', 3)]
C -> [('E', 1), ('F', 8)]
D -> [('F', 5)]
E -> [('D', 2), ('F', 6)]

Traversals

BFS: A B C D E F
DFS: A B D F E C

Explanation

- BFS (Breadth First Search):
  - Visits nodes level by level
  - Uses queue
  - Used in shortest path problems
- DFS (Depth First Search):
  - Goes deep first
  - Uses recursion/stack
  - Used in cycle detection

5. Hash Table (Separate Chaining)

Initial Table

0 : [(10, 100), (15, 150), (20, 200)]
1 : []
2 : [(7, 70), (12, 120)]
3 : []
4 : []

Operations

Get 10: 100
Get 15: 150
Get 7: 70

After Deletion

After delete 15:
0 : [(10, 100), (20, 200)]

1 : []
2 : [(7, 70), (12, 120)]
3 : []
4 : []

Explanation

- Collision occurs when multiple keys map to the same index
- Separate chaining uses lists to store multiple values
- Time Complexity:
  - Average: O(1)
  - Worst: O(n)

6. Conclusion

This project demonstrates how different data structures work in practical scenarios. BST allows efficient searching and sorted traversal, graphs help in network traversal, and hash tables provide fast data access.

These structures are widely used in databases, caching systems, and networking applications. The implementation helped in understanding both theoretical and practical aspects of data structures.